Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Prototype kalıbı: amaç ve GoF örneği (ekrandan, 00:10:00)
  - Amaç: yeni nesneleri sıfırdan, bütün üye alanlarını vererek oluşturmak yerine, temel olarak kullanılacak bir prototip nesnenin kopyasını alıp yalnız bir iki alanını değiştirerek oluşturmak.
  - Ne zaman işe yarar: nesneler "ağır sıklet" ise, çok fazla durum bilgisi varsa, kurucuya aşırı sayıda parametre geçmek gerekiyorsa.
  - GoF örneği: nota yazma programı, hazır bir grafik çerçevesi üzerine kuruluyor. Soyut Tool (Manipulate()) ve alt sınıfları RotateTool, GraphicTool; soyut Graphic (Draw(Position), Clone()) ve alt sınıfları Staff (porte) ile soyut MusicalNote'un alt sınıfları WholeNote, HalfNote. GraphicTool'dan Graphic'e "prototype" adlı ilişki vardır.
  - Sorun: çerçeve program nota sınıflarını önceden bilemez. GraphicTool.Manipulate(): `p = prototype->Clone()`, kullanıcı fareyi sürüklerken `p->Draw(new position)`, sonunda p çizime eklenir. Notaların Clone() metodu "return copy of self" yapar.
  - Alternatifler: her nota için ayrı Factory Method ya da notaları aile-üye gibi modelleyen bir Abstract Factory aşırı sayıda sınıf ve metot doğurur; bu yüzden tercih edilmez.
- Prototype kalıp yapısı (ekrandan, 00:14:00)
  - Prototype: nesnelerin kendisini klonlaması için bir arayüz sunar (soyut Clone()).
  - ConcretePrototype: kullanılacak nesneler; Clone() kendisinin kopyasını döndürür.
  - Client: bir prototipin kendisini kopyalamasını isteyerek (`p = prototype->Clone()`) yeni nesne oluşturur.
- Prototype'ın uygulanabileceği anlar (00:15:21)
  - Hangi sınıftan nesne oluşturulacağı çalışma anında belirlenecekse.
  - Ürün sınıf hiyerarşisine paralel bir fabrika hiyerarşisi kurmak istenmiyorsa.
  - Ürün sınıfının örneklerinin durum uzayı sınırlıysa (az sayıda farklı nesne olacaksa).
  - Kalıp daha çok reflection (çalışma anında bir nesnenin sınıfını, metotlarını, bağımlılıklarını sorgulama, hatta yeni özellik ekleme) ya da late binding (bir değişkenin türünün ilk atamayla belirlenmesi) desteği olmayan dillerde işe yarar.
- Shallow copy ve deep copy (00:18:48)
  - Shallow copy: yalnız ilk düzey üye alanlar kopyalanır. Deep copy: primitive olmayan üyelerin üyeleri de, nereye kadar gidiyorsa kopyalanır.
  - Deep copy çevrimsellik varsa sorun çıkarır: takım oyuncularını, oyuncu da takımını biliyorsa takım kopyalanırken aynı nesneyi tekrar kopyalamamak için nesne kimliği gibi bir denetim gerekir (ekrandan, 00:20:00; hoca Violet UML'de Takım–Oyuncu şemasını çizdi).
- Prototype örnek gerçeklemesi: oyun level editörü (ekrandan, 00:25:30, 00:28:50, 00:30:00)
  - Problem: haritaya çok çeşitli canavarlar kolayca eklenmek isteniyor, ama her canavarın HP, DEX, AGL, LUCK, CHR gibi çok sayıda durum bilgisi var. Kaynak: Head First Design Patterns.
  - «Abstract» Monster (Prototype rolü): `# HP, DEX, AGL, LUCK, CHR : int`, `# type : String`, `+ clone() : Monster {abstract}`, `+ getType() : String`, `# Monster(int, int, int, int, int)`.
  - GenericMonster (ConcretePrototype): kurucusu `super(hp, dex, agl, luck, chr)` çağırıp `type = "Generic Monster"` atar; `clone()` `new GenericMonster(HP, DEX, AGL,`

`LUCK, CHR)` döndürür.
  - MonsterRegistry: `HashMap<String, Monster> monsters; findMonster(type) monsters.get(type)` döndürür (yoksa null), `registerMonster(m) monsters.put(m.getType(), m)` yapar. MainApp istemcidir.
  - MainApp: bir GenericMonster(10, 10, 10, 10, 10) kaydedilir; "Generic Monster" bulunup klonlanır, `setHP(100)` ve `setType("Bölüm sonu canavarı")` ile değiştirilip yeniden kaydedilir.
- Singleton kalıbı (ekrandan, 00:33:30)
  - Amaç: bir sınıftan yalnız bir nesne oluşturulması ve bu nesneye her zaman aynı yoldan erişilmesi. Örnekler: yazdırma biriktiricisi, görev yöneticisi, dosya yöneticisi, pencere yöneticisi.
  - Çözüm: sınıfa kendisinin tek bir örneğinin olmasını sağlama sorumluluğunu atamak. Sınıfın kendi türünden static bir üyesi (`- uniqueInstance : Singleton {static}`) olur, kurucusu private olur, bu üye için static bir factory metodu (`+ getInstance() : Singleton {static}`) bulunur.
  - getInstance(): `if (uniqueInstance == null) { uniqueInstance = new Singleton(...) } return uniqueInstance;`.
  - Tek bir sınıf kullanıyorsa basit bir static üye yeterlidir; kalıp, tekil nesneyi farklı sınıflar kullanıyorsa ve Singleton'ın alt sınıfları oluşturulmak isteniyorsa daha yararlıdır.
- Adapter kalıbı (ekrandan, 00:40:00, 00:43:20)
  - Amaç: işlev açısından uyumlu ancak metot adları açısından uyumsuz bir istekçi sınıf ile bir sunucu sınıfı birbirleriyle tek yönlü konuşturmak.
  - Örnek: İngiliz standardındaki prize Avrupa standardındaki cihazı takmak için seyahat adaptörü kullanılır.
  - Yapı: Client, soyut Target'ın Request()'ini çağırır. Adapter, Target'tan türer ve adaptee üyesi üzerinden Adaptee'nin SpecificRequest()'ini çağırır.
- Adapter örnek gerçeklemesi: çizim programında hazır daire sınıfı (ekrandan, 00:46:40)
  - Shape (setLocation, getLocation, display, fill, setColor, undisplay) ve alt sınıfları Point, Line, Square hazır. Elde XXCircle (setLocation, getLocation, displayIt, fillIt, setItsColor, undisplayIt) var ama metot adları uymuyor ve kaynak kodu yok.
  - Çözüm: Shape'ten yeni bir Circle sınıfı türetilir, içine XXCircle üyesi konur; Circle'ın her metodu XXCircle'ın karşılık gelen metodunu çağırır. Circle Adapter, XXCircle Adaptee rolündedir.
- Bridge kalıbı: amaç ve GoF örneği (01:02:00, ekrandan 01:10:00)
  - Amaç: bir nesnenin arayüzünü (soyutlamasını) gerçeklemesinden ayırmak; böylece ikisi bağımsız olarak çeşitlenebilir.
  - GoF örneği: farklı masaüstü ortamlarını (X Window, Presentation Manager, GNOME, KDE) destekleyen bir pencere soyutlaması. Kalıtımla her ortam için ayrı pencere ve ayrıca her ortam için ayrı IconWindow sınıfı gerekir; sınıf sayısı katlanır, ortak kod paylaşılamaz, çoklu kalıtım her dilde yoktur.
  - Çözüm: soyut Window (DrawText(), DrawRect()) ve alt sınıfları IconWindow (DrawBorder()), TransientWindow (DrawCloseBox()); soyut WindowImp (DevDrawText(), DevDrawLine()) ve alt sınıfları XWindowImp, PMWindowImp. Window'dan WindowImp'e "imp" adlı ilişki vardır; DrawRect() dört kez `imp->DevDrawLine()` çağırır.
- Bridge kalıp yapısı (ekrandan, 01:14:20)
  - Abstraction (Operation(): `imp->OperationImp()`), RefinedAbstraction; Implementor (OperationImp()), ConcreteImplementorA ve ConcreteImplementorB. Soyutlama, gerçekleme türünden bir nesneyi üye alan olarak tutar.
  - Uygulanabileceği anlar: soyutlama ile gerçekleme arasındaki bağın sabit değil dinamik olması istendiğinde; soyutlamaların ve gerçeklemelerin ayrı ayrı özelleşmesi gerektiğinde.
- Bridge örnek gerçeklemesi: iki çizim kütüphanesiyle şekiller (ekrandan, 01:20:00, 01:25:40, 01:30:00, 01:32:20)
  - Problem: dikdörtgenlerin iki ailesi var; birinci aile DP1 (`draw_a_line(x1, y1, x2, y2)`), ikinci aile DP2 (`drawline(x1, x2, y1, y2)`) kütüphanesiyle çizilir.
  - İlk çözüm: soyut Rectangle (`+draw()`, `#drawLine()`), alt sınıfları V1Rectangle (DP1 kullanır) ve V2Rectangle (DP2 kullanır). drawLine protected'dır, çünkü nesnenin iç çalışmasıyla ilgilidir.
  - Daire eklenince Shape üst sınıfı ve V1Circle, V2Circle gelir; üçgen ya da DP3 eklendikçe sınıf sayısı şekil sayısı ile kütüphane sayısının çarpımı kadar büyür.

  - Bridge çözümü: soyut Shape (`+draw()`, `#drawLine()`, `#drawCircle()`, `private Drawing _dp`) ve alt sınıfları Rectangle, Circle; soyut Drawing (`drawLine()`, `drawCircle()`) ve alt sınıfları V1Drawing (DP1'i kullanır), V2Drawing (DP2'yi kullanır).
  - Shape'in kurucusu Drawing nesnesini alır; gerçeklemesi olmayan bir soyutlama olmaz. Circle, `draw()` içinde üst sınıfın `drawCircle(_x, _y, _r)` metodunu çağırır; o da `_dp.drawCircle(...)`'a iletir.
  - Yeni şekil için yalnız Shape'in bir alt sınıfı, yeni kütüphane için yalnız Drawing'in bir alt sınıfı eklenir.
- Composite kalıbı: amaç ve GoF örneği (01:35:10, ekrandan 01:40:00)
  - Amaç: hiyerarşi şeklindeki parça-bütün ilişkilerini desteklemek; istemci tek tek nesneleri ve onların oluşturduğu bileşikleri aynı biçimde görsün. Gerçek dünya örnekleri: ordu (kolordu, tümen, tugay, alay), üniversite (rektörlük, fakülte, bölüm, anabilim dalı).
  - GoF örneği: grafik çizim programında gruplama. Soyut Graphic (Draw(), Add(Graphic), Remove(Graphic), GetChild(int)); Line, Rectangle, Text ve Picture alt sınıfları. Picture'ın "graphics" adlı Graphic listesi vardır; Draw() `forall g in graphics: g.Draw()` yapar, Add(g) g'yi listeye ekler.
- Composite kalıp yapısı (ekrandan, 01:50:00)
  - Component: nesneler topluluğunun katılımcıları için ortak kullanım arayüzü sunar; bir bütünün parçaları için erişim ve düzenleme metotları (Add, Remove, GetChild) tanımlar.
  - Leaf: topluluktaki parçaları simgeler; hiyerarşinin en altındaki düğümlerdir, çocukları olamaz.
  - Composite: hiyerarşinin ara düğümleridir; "children" listesi Component türündendir, Operation() `forall g in children: g.Operation()` yapar. Farklı tür ara düğümler varsa bunlar Composite'in alt sınıfları olarak modellenir.
  - Client parçaları ve toplulukları benzer biçimde kullanabilir; slayta göre Client bir Visitor olabilir.
- Composite örnek gerçeklemesi: makine üretim fabrikası (ekrandan, 02:00:00, 02:02:20, 02:05:10, 02:10:00, 02:11:10)
  - Problem: makineler ve makine parçaları üretiliyor, ikisi de ayrı satılıyor; hangi makinenin ve alt parçalarının hangi fabrikada üretildiği sorgulanmak isteniyor. Kaynak: Design Patterns Java Workbook (değiştirilerek).
  - «abstract» ManufacturingItem: `machineID : int`, `name`, `factory : String`; getter'lar ve setFactory(); soyut getParentID(), setParentID(int), `getParts() : Iterator<ManufacturingItem>`, `addPart(part) : boolean`, `removePart(part) : boolean`.
  - Machine: `parts : ArrayList<ManufacturingItem>`; getParentID() 0 döndürür, setParentID() boştur. addPart(part) önce `part.setParentID(getMachineID())` yapar, parça zaten listedeyse false döndürür, değilse `parts.add(part)`.
  - MachinePart: `parentID : int`; getParts() null, addPart ve removePart false döndürür.
  - Report.findItems(items, factory): her öğe ve (varsa) alt parçaları için fabrika adı `compareToIgnoreCase` ile karşılaştırılır, eşleşenler numaralanıp yazdırılır, sayısı döndürülür.
  - Bursa sorgusunun çıktısı: 1. 99 Krank Şaftı KŞ-7, 2. 259 Flanş FTR-5, 3. 63 Distribütör DST-12; "3 parça bulundu."

Hocanın Özellikle Vurguladığı Kısımlar

- Prototype'ta clone() soyut bırakılır
  - Shallow mu deep mi kopya yapılacağına her somut alt sınıf kendisi karar verebilsin diye. Bütün alt sınıflarda ortak bir davranış varsa üst sınıfta varsayılan bir gövde verilebilir.
- Prototype'ta kurucuyu protected yapmak
  - Kimsenin sıfırdan, kendi istediği gibi canavar oluşturmaması, yalnız klonlama yoluyla yeni nesne oluşturulması için. Private yapılırsa alt sınıflar da erişemez.
- Prototype'ta nesneleri yöneten bir kayıt (registry) gerekir
  - GoF bunu ayrıca tartışmamış, ama hocaya göre klonlanacak nesnelerin bir yerde yönetilmesi illaki gerekecek.
- Prototype'ın kazancı ölçekte görünür
  - Tek bir canavar için registry kurmak kazanç değildir; on binlerce canavarı onlarca

parametreyle ilklendirmek yerine kopyalayıp bir iki alanı değiştirmek asıl kazançtır.
- Singleton her zaman gerekli değildir
  - Tekil veriye tek bir sınıf erişiyorsa basit bir static üye alan daha doğrudur. Bir kalıbın illaki ayrıntılı bir çözüm olması gerekmez; bazen basit fikirler günü kurtarır.
- GoF şemalarındaki kalıtım okunu UML'ye göre çizin
  - GoF zamanında UML standardı oturmamıştı; kalıtım okunun ucu üst sınıfa dayanmalıdır (ekrandan, 01:10:00).
- Neyin değiştiğini bul ve onu sarmala
  - Bridge bu ilkeyi akıllıca uygular: değişen iki şey (soyutlama ve gerçekleme) ayrı kalıtım hiyerarşilerine konur ve ikisi arasındaki bağ kalıtımla değil, üye alan (sahiplik ilişkisi) ile kurulur.
- Bridge'de soyutlamanın üst sınıfı interface olamaz
  - Shape, üye alan (_dp) ve gövdesi dolu metotlar içerdiği için soyut sınıf olmak zorundadır. Gerçekleme hiyerarşisinin üst sınıfı (Drawing) ise interface olabilir; hoca önce bunun da olamayacağını söyledi, sonra sözünü geri aldı (01:30:07, 01:31:49).
- Composite kalıtımın özüne aykırıdır
  - Leaf, çocuk yönetimiyle ilgili metotlara ihtiyaç duymadığı halde onları boş gövdeyle ya da null, 0, false döndürerek gerçeklemek zorundadır. Ayrıca programcının vermesi gereken gerçekleme kararları çoktur; kalıp gerçekten gerekiyorsa kullanılmalıdır.
- Primitive Obsession uyarısı
  - Fabrika örneğinde fabrikanın yalnız String ad olarak tutulması doğru modelleme değildir (primitive obsession); her fabrika ayrı bir sınıf olmalıydı, örneği sade tutmak için böyle yapıldı (ekrandan, 02:00:00).
- Kod tekrarı (duplicated code) kusuru ve Extract Method
  - Report.findItems'taki iki benzer blok bir kod kusurudur; ReportV2'de tekrar eden kısım ManufacturingItem parametreli ayrı bir metoda (showItem) çıkarılmıştır.

Kısa Tekrar Notları

- Prototype: yeni nesne = prototipin klonu + birkaç alan değişikliği. Roller: Prototype (soyut Clone), ConcretePrototype, Client.
- Prototype ne zaman: tür çalışma anında belli olacaksa, paralel fabrika hiyerarşisi istenmiyorsa, durum uzayı sınırlıysa; reflection ya da late binding yoksa.
- Shallow copy ilk düzeyi, deep copy bütün alt üyeleri kopyalar; döngüsel ilişkilerde deep copy dikkat ister.
- Level editörü: Monster (soyut clone, protected kurucu), GenericMonster, MonsterRegistry (HashMap, findMonster, registerMonster).
- Singleton: static uniqueInstance, private kurucu, static getInstance (null ise oluştur, sonra döndür).
- Adapter: Client → Target.Request(); Adapter.Request() → adaptee.SpecificRequest(). Örnek: Circle (Adapter) → XXCircle (Adaptee).
- Bridge: Abstraction ve Implementor ayrı hiyerarşiler, arada "imp" üyesi. Örnek: Shape/Rectangle/Circle ve Drawing/V1Drawing/V2Drawing.
- Bridge'siz tasarımda sınıf sayısı (şekil × kütüphane), Bridge ile (şekil + kütüphane) olur.
- Composite: Component (Operation, Add, Remove, GetChild), Leaf, Composite (children listesi, işlemi bütün çocuklara iletir).
- Fabrika örneği: ManufacturingItem, Machine (Composite), MachinePart (Leaf); Bursa sorgusu 3 parça bulur.

Detaylı Açıklamalar

- Prototype'ta GraphicTool'un çalışması
  - Araç çubuğundaki her düğme bir Tool'dur (döndürme, boyutlandırma, taşıma, yeni şekil ekleme). GraphicTool kendi prototip nesnesinin kopyasını alır, kullanıcı fareyi sürükledikçe yeni nesneyi o konumda çizer, bırakınca çizime ekler. Çerçeve yalnız Graphic'in Draw ve Clone metotlarına, araçlar da Manipulate metoduna dayandığı için grafik çerçevesi daha önce düşünülmemiş alanlara genişletilebilir.
  - Hoca, Clone()'un dönüş türünün slaytta yazılmadığını belirtip şemaya `Clone(): Graphic` olarak ekledi (ekrandan, 00:10:00).

- Prototype'ta late binding örneği
  - Kimlik bilgisi olarak vatandaşlardan kimlik numarası, yabancılardan pasaport numarası alınacaksa, oluşturulacak kişinin hangisi olduğu ancak verilen veriye göre çalışma anında belli olur. Dil late binding'i desteklemiyorsa bu tür durumlar Prototype ile çözülebilir.
- Deep copy'de çevrimsellik
  - Takım–oyuncu ilişkisinde takım kopyalanırken oyuncular da kopyalanır, ama oyuncular takımı da bilir; takımı ikinci kez kopyalamamak için nesne kimliği ya da referans denetimi gerekir. Hoca, altyapı takımı gibi ek ilişkilerle şemanın daha da karmaşıklaşabileceğini, Hibernate gibi hazır yapıların bu işi kolaylaştırabileceğini söyledi.
- Level editörünün kodu (ekrandan, 00:28:50, 00:30:00)

```
package dp.prototype.example;
public class GenericMonster extends Monster {
    public GenericMonster(int hp, int dex, int agl, int luck, int chr) {
        super(hp, dex, agl, luck, chr);
        type = "Generic Monster";
    }
    public Monster clone( ) {
        return new GenericMonster(HP, DEX, AGL, LUCK, CHR);
    }
}
public class MonsterRegistry {
    private HashMap<String, Monster> monsters;
    public MonsterRegistry() { monsters = new HashMap<String,Monster>(); }

    public Monster findMonster( String type ) {
        return monsters.get( type );
    }
    public void registerMonster( Monster m ) {
        monsters.put( m.getType(), m );
    }
}
public class MainApp {
    public static void main(String[] args) {
        MonsterRegistry reg = new MonsterRegistry();
        reg.registerMonster( new GenericMonster(10, 10, 10, 10, 10) );
        Monster mySpecialMonster =
                reg.findMonster("Generic Monster").clone();
        mySpecialMonster.setHP(100);
        mySpecialMonster.setType("Bölüm sonu canavarı");
        reg.registerMonster(mySpecialMonster);
    }
}
```

  - Ders sonrası not: GenericMonster.clone() type alanını kopyalamaz; kurucu her zaman "Generic Monster" atar. Bu yüzden kayıttaki "Bölüm sonu canavarı" klonlanırsa klonun türü yeniden "Generic Monster" olur. Hoca bu ayrıntıya değinmedi.
- Singleton ile static üye farkı
  - İlk akla gelen çözüm, tek olacak nesneyi bir sınıfın static üyesi yapmaktır. Ama nesneyi farklı sınıflar kullanıyorsa üyenin hangisine konacağı sorun olur ve değişiklikte birden fazla sınıfa dokunmak gerekir. Singleton sorumluluğu nesnenin kendi sınıfına verir; nesneye erişmek için başka bir sınıfa gitmek gerekmez ve Singleton'ın alt sınıfları da tekil kalır. Hoca, ileride görülecek State ve Strategy kalıplarındaki nesnelerin de Singleton adayı olduğunu söyledi.
- Bridge'de kalıtım çözümünün tıkanması

  - Pencereyi simge durumuna küçültmenin yanında sistem tepsisinde (saatin olduğu alanda) bir sürümünü göstermek gibi yeni bir gereksinim gelince IconWindow sınıfı eklenir; ama her ortam için ayrı bir IconWindow alt sınıfı gerekir (dört ortam için dört ek sınıf). Abstract Factory'de aile üyeleri (düğme, radyo düğmesi) işlevce birbirinden ayrıktır; burada ise pencere ile simge penceresi arasında paylaşılan ortak kod vardır, bu yüzden Bridge daha uygundur.
  - Bridge'de IconWindow'a bir XWindowImp nesnesi verilince X Window altında çalışan bir simge penceresi olur; Presentation Manager'a taşımak için yalnız ona PMWindowImp nesnesi verilir.
  - Hoca, ileride görülecek Strategy kalıbının Bridge'e benzediğini, ama Strategy'de soyutlamanın alt sınıflarının olmadığını belirtti.

- Bridge örneğinin kodu (ekrandan, 01:30:00, 01:32:20)

```
package bridge.solution2;
public abstract class Shape {
   public abstract void draw( );
   private Drawing _dp;
   public Shape (Drawing dp) { _dp= dp; }
   protected void drawLine(double x1, double y1,
     double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
   protected void drawCircle(double x,double y,
     double r ) {_dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
   private double _x, _y, _r;
   public Circle (Drawing dp,
               double x,double y,double r) {
      super( dp) ;
      _x= x; _y= y; _r= r ;
   }
   public void draw () { drawCircle(_x,_y,_r); }
}
public abstract class Drawing {
   public abstract void drawLine( double x1, double y1,
             double x2, double y2 );
   public abstract void drawCircle (double x,double y,
             double r);
}
public class V1Drawing extends Drawing {
   public void drawCircle(double x, double y, double r) {
      DP1.draw_a_circle(x,y,r);
   }
   public void drawLine(double x1, double y1,
             double x2, double y2) {
      DP1.draw_a_line(x1,y1,x2,y2);
   }
}
```

  - Alt sınıf nesneleri gerçeklemeye doğrudan dokunmaz, yalnız üst sınıfın protected metotları üzerinden iş yaptırır. Rectangle da aynı biçimde köşe koordinatlarını alır ve dört drawLine çağrısıyla çizilir. V2Drawing her işi DP2 kütüphanesine yaptırır.

- Composite'te ebeveyn bilgisi
  - İşlem yalnız kökten gönderilecekse ilişki Composite'ten Component'e tek yönlüdür. Her düğüm ebeveynini bilmek isterse yaprağa Composite türünden bir parent üyesi eklenir (Component değil, çünkü bir çizgi bir daireye ebeveyn olamaz).
  - Herhangi bir yapraktan gelen işlemin bütün bileşenlere iletilmesi istenirse: kökün parent'ı null olur, yaprağa gelen ileti önce köke iletilir, kökten bütün düğümlere dağıtılır; sonsuz döngüye girmemek için her düğümde "isVisited" gibi bir alan tutulur. Hoca bunun tasarımını ve kodlamasını öğrencilere bıraktı ve normalde istemcinin

yalnız kökle konuşmasının bu ek yükü ortadan kaldırdığını söyledi.

- Composite'te düğüm silme kararları
  - Bir ara düğümü ebeveyni mi yoksa kök mü silmeli? Silinen düğümün çocukları da silinecek mi, başka bir Composite'e mi aktarılacak, yoksa silinenin ebeveynine mi taşınacak? Hangi ağaç veri yapısı kullanılacak? Problem bu konuda bir şey söylemiyorsa programcı yeni kurallar koymak zorunda kalır.
- Fabrika örneğindeki tasarım kararları (ekrandan, 02:00:00, 02:02:20)
  - Hiyerarşi iki düzeylidir (makine ve makine parçası). parentID yalnız MachinePart için anlamlıdır; ManufacturingItem'a konsaydı Machine'e de anlamsız biçimde aktarılacaktı. Hoca kalıtımın özüne ters davranışı en aza indirmek için parentID'yi üst sınıfa koymadı, getParentID ve setParentID'yi soyut bıraktı. Machine'de bu metotlar soyut bırakılamaz, yoksa Machine nesnesi oluşturulamazdı; bu yüzden getParentID() 0 döndürür, setParentID() hiçbir şey yapmaz.

```
public Machine( int machineID, String name, String factory ) {
    super(machineID, name, factory);
    parts = new ArrayList<ManufacturingItem>();
}

public int getParentID() { return 0; }
public void setParentID(int parentID) { }

public boolean addPart( ManufacturingItem part ) {
    part.setParentID(getMachineID());
    for( ManufacturingItem aPart : parts )
        if( aPart == part )
            return false;
    return parts.add(part);
}
public boolean removePart( ManufacturingItem part ) { return parts.remove(part); }
public Iterator<ManufacturingItem> getParts( ) { return parts.iterator(); }
```

  - MachinePart'ta getParts() null döndürür, addPart ve removePart false döndürür; hoca her zaman başarılı sayılıp true da döndürülebileceğini söyledi.
- Rapor ve çalıştırma (ekrandan, 02:05:10, 02:10:00, 02:11:10)
  - Katalog dört elemanlıdır: items[0] Krank Şaftı KŞ-7 (99, Bursa), items[1] Jeneratör JMX-99 (135, İstanbul; parçaları Flanş FTR-5 (259, Bursa), Kasnak KS-9 (378, İstanbul), Rulman RL-3 (196, Ankara)), items[2] Distribütör DST-12 (63, Bursa), items[3] Pnömatik Vana VPN-7 (82, Ankara).
  - `Report.findItems(items, "Bursa")` jeneratörün kendisini bulmaz (İstanbul), ama Bursa'da üretilen Flanş parçasını bulur. Sonuç: Krank Şaftı, Flanş, Distribütör; "3 parça bulundu."
  - addPart parçanın parentID'sini zaten atadığı için parça kurucusuna verilen 135 değeri şart değildi; hoca okunaklı olsun diye vermiş.
  - İki ayrı döngüden de kurtulmak için ileride görülecek Visitor kalıbı kullanılabilir, ama o da başka karmaşıklıklar getirir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.